

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..18

SETUP

 Copy as Markdown

Set up the authentication module

Serverpod comes with built-in user management and authentication. It is possible to build a [custom authentication implementation](#), but the recommended way to authenticate users is to use the `serverpod_auth_idp` module. The module lets users sign in through identity providers. An identity provider is a service that verifies who the user is, such as email with password, Google, or Apple. The module also handles basic user information, such as user names and profile pictures.

This page walks through the manual setup: install the module on the server, the client, and the app, then wire up the sign-in UI. New projects created with `serverpod create` already include this setup. For that path, see [Get started](#).



Sign In with email



[Forgot password?](#)

Sign in

Don't have an account? [Sign up](#)

or continue with



Continue with Google



Continue with Apple

Server setup

Add the authentication module as a dependency to the server project's `pubspec.yaml`. Use the same version numbers as for Serverpod itself for all dependencies.

```
dependencies:
  ...
  serverpod_auth_idp_server: 4.0.0
```

The `serverpod_auth_idp_server` package contains all components required to configure authentication services.

Configure authentication services

In your main `server.dart` file, configure the authentication system using the `pod.initializeAuthServices()` extension method.

The configuration needs at least one token manager. A token manager issues and validates the tokens that keep users signed in after they authenticate. The example below uses JWT tokens and reads its secrets from your password store. The [token manager configuration](#) section explains the options.

```
import 'package:serverpod_auth_idp_server/core.dart';

import 'src/generated/serverpod.dart';

void run(List<String> args) async {
  final pod = Serverpod(args);

  // Set up authentication services
  // The `pod.getPassword()` will get the value from `config/passwords.yaml`.
  pod.initializeAuthServices(
    tokenManagerBuilders: [
      JwtConfig(
        // Pepper used to hash the refresh token secret.
        refreshTokenHashPepper: pod.getPassword('jwtRefreshTokenHashPepper')!,
        // Algorithm used to sign the tokens (`hmacSha512`, `hmacSha256` or
        // `ecdsaSha512`).
        algorithm: JwtAlgorithm.hmacSha512(
          // Private key to sign the tokens. Must be a valid HMAC SHA-512 key.
          SecretKey(pod.getPassword('jwtHmacSha512PrivateKey')!),
        ),
      ),
    ],
  );

  await pod.start();
}
```

See [storing secrets](#) for what the pepper in the example is and where to keep it.

JWT-based authentication also needs a refresh endpoint. The app calls it to renew expired access tokens without asking the user to sign in again. Extend the abstract endpoint to expose it on the server. Create the file anywhere under your server's `lib/` directory, for example `<project>_server/lib/src/endpoints/`. The generator picks it up:

```
import 'package:serverpod_auth_idp_server/core.dart' as core;

class RefreshJwtTokensEndpoint extends core.RefreshJwtTokensEndpoint {}
```

Token manager configuration

Token managers issue and validate the tokens that keep users signed in. Configure at least one. The first one in the list is the primary manager, which issues new tokens. Any others are used to validate and manage the tokens they issued.

Serverpod provides two built-in token manager builders:

- `JwtConfig` to use JWT-based authentication. See [JWT token manager](#) for details.
- `ServerSideSessionsConfig` to use server-side sessions authentication. See [Server-side sessions token manager](#) for details.

For more details on how to configure token managers or create custom ones, see the dedicated [token managers](#) documentation.

Identity providers configuration

Identity providers handle authentication with different methods (Email, Google, Apple, etc.). Each provider has its own configuration:

- **Email**: sign-up and sign-in with email and password.
- **Anonymous** (experimental): accounts without any credentials, for trying the app before registering.
- **Google, Apple, Facebook, GitHub**, and **Microsoft**: sign-in with the respective account.
- **Firebase**: reuse Firebase Authentication, including its phone and social sign-ins.
- **Passkey** (experimental): passwordless sign-in with passkeys.
- **Custom providers**: build your own, including OAuth2-based ones.

The list of identity providers keeps growing. If you want to contribute a new provider, see the [contribution guidelines](#).

By default, endpoints for all providers are disabled. To enable a provider:

1. Pass its config object to the `identityProviderBuilders` parameter of the `pod.initializeAuthServices()` method.

```
pod.initializeAuthServices(  
  tokenManagerBuilders: [  
    JwtConfigFromPasswords(),  
  ],  
  identityProviderBuilders: [  
    EmailIdpConfig( /* configuration options */ ),  
  ],  
);
```

TIP

Some providers need credentials from an external service, such as the Google client secret. The provider's config object takes these as required parameters.

2. Extend the identity provider abstract endpoint.

```
import 'package:serverpod_auth_idp_server/providers/email.dart';  
  
class EmailIdpEndpoint extends EmailIdpBaseEndpoint {}
```

3. Start the server with `serverpod start`. It generates the client code and endpoint methods for the provider, then runs the server with hot reload.

```
$ serverpod start
```

4. Create and apply the migration that initializes the database for the provider. In the `serverpod start` terminal, press **M**. The migration is created and applied in one step. If applying fails, press **A** to retry it.

INFO

If this is the first time creating migrations after adding the module, besides the provider tables, all authentication module tables will also be created. More detailed migration instructions can be found in the [migration guide](#).

Storing secrets

A pepper is a secret string mixed into values before they are hashed, so stored hashes cannot be brute-forced from a database leak alone. It is one application-wide secret, kept outside the database. Peppers and private keys should be stored securely. The example above uses `pod.getPassword()` which reads from your

`config/passwords.yaml` file or environment variables in the format `SERVERPOD_PASSWORD_<key> 'value'`.

Add secrets to `config/passwords.yaml`:

```
development:
  serverSideSessionKeyHashPepper: 'your-session-pepper-here'
  jwtRefreshTokenHashPepper: 'your-refresh-token-pepper-here'
  jwtHmacSha512PrivateKey: 'your-private-key-here'
  emailSecretHashPepper: 'your-email-pepper-here'
  googleClientSecret: '{"web":{"client_id":"...", "client_secret":"...", "redirect_uris":
["..."]}}'
# ... other secrets
```

Or use environment variables in the expected format:

```
export SERVERPOD_PASSWORD_serverSideSessionKeyHashPepper='your-session-pepper-here'
export SERVERPOD_PASSWORD_jwtRefreshTokenHashPepper='your-refresh-token-pepper-here'
export SERVERPOD_PASSWORD_jwtHmacSha512PrivateKey='your-private-key-here'
export SERVERPOD_PASSWORD_emailSecretHashPepper='your-email-pepper-here'
export SERVERPOD_PASSWORD_googleClientSecret='{"web":
{"client_id":"...", "client_secret":"...", "redirect_uris":["..."]}}'
# ... other secrets
```

INFO

Builders that need secrets have a `FromPasswords` variant that reads them from well-known key names, so you do not need to call `pod.getPassword()` yourself. Any other configuration options are still passed as parameters. For example:

```
final jwtConfig = JwtConfigFromPasswords();
final serverSideSessionsConfig = ServerSideSessionsConfigFromPasswords();
final emailIdpConfig = EmailIdpConfigFromPasswords();
final googleIdpConfig = GoogleIdpConfigFromPasswords();
final appleIdpConfig = AppleIdpConfigFromPasswords();
final passkeyIdpConfig = PasskeyIdpConfigFromPasswords();
```

WARNING

Never commit `config/passwords.yaml` to version control. Be sure to add it to your `.gitignore` file. Prefer environment variables or secure secret management in production.

Client setup

The client is the generated Dart package that your app uses to call the server (the `_client` package in your project). Add the `serverpod_auth_idp_client` package to its `pubspec.yaml`. Use the same version numbers as for Serverpod itself for all dependencies.

```
dependencies:
  ...
  serverpod_auth_idp_client: 4.0.0
```

App setup

First, add these packages to your app's `pubspec.yaml` file. Some providers, such as Facebook and Firebase, need an extra package, which their setup page names.

```
dependencies:
  flutter:
    sdk: flutter
  serverpod_auth_idp_flutter: 4.0.0
  serverpod_flutter: 4.0.0
  your_client:
    path: ../your_client
```

Next, you need to set up a `FlutterAuthSessionManager`, which keeps track of the user's authentication state. It handles authentication tokens, token storage and refresh, and user session management.

```
import 'package:flutter/material.dart';
import 'package:serverpod_flutter/serverpod_flutter.dart';
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';
import 'package:your_client/your_client.dart';

late Client client;

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  const serverUrl = 'http://localhost:8080/';

  // Create the client with the auth session manager
  client = Client(serverUrl)
    ..connectivityMonitor = FlutterConnectivityMonitor()
    ..authSessionManager = FlutterAuthSessionManager();

  // Initialize authentication (restores session from storage and validates)
  await client.auth.initialize();

  runApp(MyApp());
}
```

The `FlutterAuthSessionManager` provides useful properties and methods for managing authentication state.

WEB APPS

On the web, enable [cookie-based authentication](#) so sign-in tokens are kept in `httpOnly` cookies instead of JavaScript-readable storage.

TIP

The `client.auth` getter is a shortcut for `client.authSessionManager`. If your project defines its own endpoint class named `AuthEndpoint`, the generated client uses the `auth` name for that endpoint instead. In that case, call `client.authSessionManager.initialize()` in the example above.

Initialize authentication

The `initialize()` method restores any existing session from storage and validates it with the server. It should be called when your app starts:

```
await client.auth.initialize();
```

This is equivalent to calling `restore()` followed by `validateAuthentication()`. If the authentication is no longer valid, the user is automatically signed out.

See [Client-side authentication](#) for more details on how to interact with the authentication state from the client.

NOTE

macOS apps need a Keychain Sharing entitlement before authentication sessions can be stored. See [Set up authentication on macOS](#).

Web callback page (`auth.html`)

NOTE

You only need this if your app targets the **web** platform and uses an identity provider that signs the user in through an OAuth2 redirect. That includes **GitHub**, **Microsoft**, **Google** on web, and custom OAuth2-based providers. Skip this section if your app does not target web, or if it only uses email, anonymous, passkey, Apple, Facebook, or Firebase sign-in.

When the user finishes signing in at the provider's page (for example, `accounts.google.com`), the provider redirects the browser to a URL on your site with the sign-in result attached. Your Flutter app cannot

receive that redirect directly because the browser navigates fully away from it. The `auth.html` file is a small static page that catches the redirect, reads the result, and hands it back to your running Flutter app through `postMessage` (or `localStorage`, depending on how the sign-in was launched).

You create one `auth.html` and share it across every identity provider that needs it.

You have two ways to deliver it.

If Serverpod serves your Flutter web app, register the `FlutterWebAuth2CallbackRoute` from `serverpod_auth_idp_server` on your web server and point the provider at that route. The page posts the result back to its own origin, so the browser only delivers it when your app is served from that same origin (same scheme, host, and port).

Otherwise, host the file yourself. In your Flutter project's `web/` folder, add a file named `auth.html` with this content, which is identical to what the route serves:

```
<!DOCTYPE html>
<title>Authentication complete</title>
<p>Authentication is complete. If this does not happen automatically, please close the
window.</p>
<script>
  function postAuthenticationMessage() {
    const message = {
      'flutter-web-auth-2': window.location.href
    };

    if (window.opener) {
      window.opener.postMessage(message, window.location.origin);
      window.close();
    } else if (window.parent && window.parent !== window) {
      window.parent.postMessage(message, window.location.origin);
    } else {
      localStorage.setItem('flutter-web-auth-2', window.location.href);
      window.close();
    }
  }

  postAuthenticationMessage();
</script>
```

When you set up a provider that uses this callback, you will register the full URL of `auth.html` in two places, and they must match exactly:

- **In the provider's OAuth client configuration:** for example, **Authorized redirect URIs** in Google Cloud Console, or **Authorization callback URL** in a GitHub OAuth app.
- **In the Flutter sign-in initializer**, via the `redirectUri` argument (e.g., `client.auth.initializeGoogleSignIn(..., redirectUri: ...)`).

When you host the file yourself, the URL is your Flutter web app's origin plus `/auth.html`. For example, `http://localhost:49660/auth.html` during local development or `https://yourdomain.com/auth.html` in production. When the server serves the page, the URL is that same origin plus the path you registered the route at, for example `https://yourdomain.com/auth/callback`. The provider's setup page walks through the exact values for that provider.

Present the authentication UI

The `serverpod_auth_idp_flutter` package provides a `SignInWidget` that automatically detects enabled identity providers and displays the appropriate sign-in options.

```
import 'package:flutter/material.dart';
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';
import 'package:your_client/your_client.dart';

class SignInPage extends StatelessWidget {
  final Client client;

  const SignInPage({required this.client, super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SignInWidget(
        client: client,
        onAuthenticated: () {
          // Do something when the user is authenticated.
          //
          // NOTE: You should not navigate to the home screen here, otherwise
          // the user will have to sign in again every time they open the app.
        },
        onError: (error) {
          // Handle errors
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text('Error: $error')),
          );
        },
      ),
    );
  }
}
```

This widget is a convenient way to use identity providers out-of-the-box, but you can also fully customize it or replace it with your own implementation. See the [UI components](#) documentation for more details.

Updating the UI based on authentication state

Do not navigate to another screen from the `onAuthenticated` callback, or the user will have to sign in again every time they open the app. Instead, listen to authentication state changes with the

`authInfoListenable` getter and switch screens based on the state. See [Client-side authentication](#) for details.

Related

- [Get started](#): the quick path for projects created with `serverpod create`.
- [The basics](#): how authentication works on the server and in the app.
- [Token managers](#): choose between JWT and server-side sessions.
- [Web setup](#): keep web sign-in tokens in httpOnly cookies.
- [UI components](#): customize or replace the sign-in UI.